

Criterion C: Development

Contents

1. Multi-threading	1
2. Stacks	1
3. Password hashing	2
4. Form handling.....	2
5. Websocket communication	3
6. Asynchronous programming	4
7. User authentication.....	5
8. File manipulation	6
9. Anonymous functions / lambda functions	7
10. Error handling.....	7
11. Rest API	8
12. Generating HTML chat messages	9
13. Template generation	9

1. Multi-threading

Due to the resource intensive nature of the program (processing live video while also providing an HTTP server and live chat), it was split into two threads: one for video processing and another for handling all web requests and serving them. This method allows the program to run efficiently as the two areas do not share a core and as such do not interrupt each other when performing their specific tasks.

```
let g_thread: Result<JoinHandle<>>, Error> = thread::Builder::new().name(String::from("gstreamer")).spawn(move || {
    stream::initStream(rx);
});
if g_thread.is_err() {
    println!("Error while starting Gstreamer thread: {}", g_thread.err().unwrap().to_string());
    std::process::exit(1);
}
let g_thread: &Thread = g_thread.unwrap().thread();
```

2. Stacks

As a way to traverse the filesystem during the setup of the streaming thread, stacks are used as they provide an easy way to traverse said filesystem without having to worry

about special symbols such as “/” or “\” and having to escape them in order to avoid errors as the conversion occurs in the background. Additionally, the ability to change the stack with very few operations proved to be very useful.

```
let mut playlist_path: PathBuf = env::current_dir().unwrap();
playlist_path.push(path: "files");
playlist_path.push(path: "hls");
playlist_path.push(path: "playlist.m3u8");

playlist_path.pop();
playlist_path.push(path: "segment%05d.ts");

hls_sink.set_property(property_name: "location", value: playlist_path.to_str().unwrap());
```

3. Password hashing

In order to provide adequate security of user information, hashing (specifically using the bcrypt algorithm) was used in order to prevent the passwords from being readable in the event of a data breach or data being accidentally exposed. In order for a user to login the inputted password is hashed and compared with the already existing hashed password in the database, thus ensuring security.

```
26 const SALT: [u8; 16] = *b"ILOVECOMPUTERSCI";
27 const ROUNDS: u32 = 4;
```

```
if queryData.get::<String, usize>(2) == bcrypt::hash_with_salt(data.password, ROUNDS, SALT)
.unwrap().format_for_version(bcrypt::Version::TwoA) {
```

```
let m: Result<SqliteQueryResult, ...> = sqlx::query(sql::CREATE_USER)
    .bind(&userid)
    .bind(&data.username)
    .bind(bcrypt::hash_with_salt(&data.password.as_bytes(), ROUNDS, SALT)
        .unwrap().format_for_version(bcrypt::Version::TwoA))
    .execute(&*&db).await;
```

4. Form handling

As a way of making the handling of client requests easier, HTTP forms were used with a standard structure which makes it easy to handle for the program and allows it to process the data immediately without having to worry about decoding the data into the appropriate format. All forms make use of the “LoginData” struct which is used mainly for login and registration however it was also repurposed for tasks such as administration.

```

44 struct LoginData {
45     username: String,
46     password: String
47 }

```

```

125 #[post("/admin/login", data = "<data>")]
126 async fn adminlogin_post(jar: &CookieJar<'_>, db: &Logs, data: Form<LoginData>) {
127     let data: LoginData = data.into_inner();
128     if data.username == "admin" && data.password == "admin" {

```

In this case, the “LoginData” struct is used in order to carry stream title and description.

```

#[post("/admin/info", format = "application/x-www-form-urlencoded", data = "<data>")]
async fn admin_infochange(data: Form<LoginData>, stream_info: &State<Mutex<(String, String)>>, _admin: AdminAccess<'_>)] -> Status {
    let stream_info: &Mutex<(String, String)> = stream_info.inner();
    let data: LoginData = data.into_inner();
    if data.username.len() > 64 {
        return Status::PayloadTooLarge;
    }
}

```

```

254 #[post("/login", data = "<data>")]
255 async fn postLogin(db: &Logs, data: Form<LoginData>, cookies: &CookieJar<'_>) -> Status {
256     let data: LoginData = data.into_inner();
257     let queryData: Result<SqliteRow, Error> = sqlx::query(sql::GET_USER_BYNAME).bind(&data.username).fetch_one(&**db).await;

```

```

283 #[post("/register", data = "<data>")]
284 async fn postRegister(db: &Logs, data: Form<LoginData>, cookies: &CookieJar<'_>) -> (Status, String) {
285     let data: LoginData = data.into_inner();
286     let query_data: Result<SqliteRow, Error> = sqlx::query(sql::GET_USER_BYNAME)
287         .bind(&data.username)
288         .fetch_one(&**db).await;

```

5. Websocket communication

Websockets were used in order to facilitate real-time communication between the client and the server as a way of implementing the live chat. The communication works by first authenticating the user and then upgrading the HTTP protocol into the websocket protocol, which then lets the user communicate with the server, who's messages are then passed on to other websocket connections.

```

Ok(ws.channel(move |mut stream| Box::pin(async move {
    let mut rx= queue.subscribe();
    let username = username;
    loop {
        select! {
            msg = stream.next() => {
                if msg.is_none() {
                    return Ok(());
                }
            }
            let msg = msg.unwrap().unwrap();
            match msg {
                Message::Text(text) => {
                    let mut content = String::from(&username);
                    content.push_str(": ");
                    content.push_str(&text);
                    let _ = queue.send(content);
                }
                Message::Ping(p) => {
                    let _ = stream.send(Message::Pong(vec![1])).await;
                }
                _ => {}
            }
        }
        rxv = rx.recv() => {
            match rxv {
                Ok(s) => {
                    let _ = stream.send(Message::text(s)).await;
                }
                _ => {}
            }
        }
    }
})
}
}
}
}

```

6. Asynchronous programming

In order to handle different tasks happening at the same time, asynchronous programming was used as a method of awaiting on those tasks and acting on the one which finishes sooner. The method was used in the aforementioned websocket communication section in order to await for messages from either the connected client or for messages from other websocket channels. In the code the “select!” block is a special block which contains the two tasks which need to be awaited and performs actions regarding whichever one finishes sooner.

```

Ok(ws.channel(move |mut stream| Box::pin(async move {
    let mut rx = queue.subscribe();
    let username = username;
    loop {
        select! {
            msg = stream.next() => {
                if msg.is_none() {
                    return Ok(());
                }
                let msg = msg.unwrap().unwrap();
                match msg {
                    Message::Text(text) => {
                        let mut content = String::from(&username);
                        content.push_str(": ");
                        content.push_str(&text);
                        let _ = queue.send(content);
                    }
                    Message::Ping(p) => {
                        let _ = stream.send(Message::Pong(vec![1])).await;
                    }
                    _ => {}
                }
            }
            recv = rx.recv() => {
                match recv {
                    Ok(s) => {
                        let _ = stream.send(Message::text(s)).await;
                    }
                    _ => {}
                }
            }
        }
    }
})
}
}
}
}

```

7. User authentication

When a user logs in their account is linked to a special randomly generated “session” cookie which gets stored in the database and is verified by the server on each page to check whether the user is logged in or not. Additionally, the session cookie has a set expiration time, denoted by the “SESSIONTIME” constant.

```

28  const SESSIONTIME: i64 = 115200;

```

```

264  if queryData.get::(<String, usize>(2) == bcrypt::hash_with_salt(data.password, ROUNDS, SALT)
265  .unwrap().format_for_version(bcrypt::Version::TwoA) {
266      let cookieData: String = util::generateCookie();
267      cookies.add(Cookie::build("session", cookieData.clone()))
268      .same_site(rocket::http::SameSite::Strict).expires(None));
269
270      let _ = sqlx::query(sql::CREATE_SESSION)
271      .bind(&cookieData)
272      .bind(queryData.get::(<&str, usize>(0))
273      .bind(util::timeNow() as i64 + SESSIONTIME)
274      .execute(&**db);
275      return Status::Ok;
276  }

```

Additionally, if a user fails to authenticate on special admin pages, which are only displayed to logged in administrators, they will be automatically redirected to a login

page. The first “[get...]” specifies the response if the user is authenticated successfully, whereas the “[get...]” with “rank = 2” responds only when the user could not be authenticated successfully.

```
#[get("/admin")]
async fn admin(jar: &CookieJar<'_>, _admin: AdminAccess<'_>, stream_info: &State<Mutex<(String,
String)>>, stream_key: &State<Mutex<String>>) -> Template {
    let stream_info = stream_info.inner().lock().unwrap();
    let stream_key = stream_key.inner().lock().unwrap();
    return Template::render("admin", context! {stream_title: stream_info.0.clone(), stream_description:
stream_info.1.clone(),
    stream_key: stream_key.clone()
});
}
#[get("/admin", rank = 2)]
async fn admin2() -> Redirect {
    Redirect::to(uri!("/admin/login"))
}
```

8. File manipulation

In order to fulfill success criteria number x, the program performs reading and writing operations on a CSS file in order to change its contents and fulfill the success criteria.

```
85 #[put("/admin/css", format = "text/css", data = "<input>")]
86 async fn admincss_put(jar: &CookieJar<'_>, input: String, _admin: AdminAccess<'_>) -> Status {
87     let mut dir: PathBuf = PathBuf::from(util::get_workdir());
88     dir.push("files/css/index.css");
89     let file: Result<File, Error> = std::fs::OpenOptions::new().truncate(true).write(true).open(&dir);
90     let mut file: File = file.unwrap();
91     file.write_all(&input.as_bytes()).unwrap();
92     file.flush().unwrap();
93     Status::Ok
94 }
```

In case the administrator wants to go back to the default CSS file, the program will erase the customized CSS file titled “index.css” and instead will replace it with the data contained in “index2.css” which contains the default data.

```
99 #[delete("/admin/css")]
100 async fn admincss_delete(jar: &CookieJar<'_>, db: &Logs, _admin: AdminAccess<'_>) -> Status {
101     let mut dir: PathBuf = PathBuf::from(util::get_workdir());
102     dir.push("files/css/index2.css");
103     let mut readvec: Vec<u8> = vec![];
104     std::fs::File::open(&dir).unwrap().read_to_end(&mut readvec).unwrap();
105     dir.pop();
106     dir.push("index.css");
107     let mut file: File = std::fs::OpenOptions::new().read(true).write(true).truncate(true).open(&dir)
108     .unwrap();
109     file.write_all(&readvec).unwrap();
110
111     Status::Ok
112 }
```


9. Anonymous functions / lambda functions

The frontend code works mostly in terms of events which have to be triggered explicitly by the user such as clicking a button or writing some text. As such in order to act on those events they have to be registered with the browser and provided an anonymous function which provides the desired functionality to the site.

```
35  function onEnter(ev) {
36      ev.preventDefault()
37      if (socket != undefined) {
38          socket.send(document.getElementById("chatinput").value)
39          document.getElementById("chatinput").value = ""
40      }
41  }
```

Aside from user triggered events, lambda functions are also present for other cases, such as receiving a websocket message.

```
socket = new WebSocket("/chat")
socket.onmessage = (ev) => {
    let split = ev.data.toString().split(":")
    addMessage(split[0], split[1])
}
socket.onopen = (ev) => {
    addMessage("SYSTEM", "Welcome to the live chat! Remember to follow the rules!")
}
socket.onclose = (ev) => {
    console.warn(ev)
}
socket.onerror = (ev) => {
    console.warn(ev)
}
```

10. Error handling

In order to prevent failures and panics, error handling was implemented in order to handle potential problems with data and provide alternative behaviour such as shutting down the program while reporting an appropriate error. This helps avoid situations such as parts of the program not working correctly or preventing data from being lost if a user attempts to use the website when its malfunctioning.

```
363  if g_thread.is_err() {
364      println!("Error while starting Gstreamer thread: {}", g_thread.err().unwrap().to_string());
365      std::process::exit(1);
366  }
```

11. Rest API

As a way to accommodate required features such as custom CSS or the ability to change stream title and description, a REST API was created with multiple endpoints and the ability to handle different types of HTTP requests such as GET, POST, PUT and DELETE. This was created in order to make programming the front-end less complicated and clearer. Additionally, it separates different functionalities from each other.

PUT method endpoint.

```
85 #[put("/admin/css", format = "text/css" , data = "<input>")]
86 async fn admincss_put(jar: &CookieJar<'_>, input: String, _admin: AdminAccess<'_>) -> Status {
87     let mut dir: PathBuf = PathBuf::from(util::get_workdir());
88     dir.push("files/css/index.css");
89     let file: Result<File, Error> = std::fs::OpenOptions::new().truncate(true).write(true).open(&dir);
90     let mut file: File = file.unwrap();
91     file.write_all(&input.as_bytes()).unwrap();
92     file.flush().unwrap();
93     Status::Ok
94 }
```

DELETE method endpoint.

```
99 #[delete("/admin/css")]
100 async fn admincss_delete(jar: &CookieJar<'_>, db: &Logs, _admin: AdminAccess<'_>) -> Status {
101     let mut dir: PathBuf = PathBuf::from(util::get_workdir());
102     dir.push("files/css/index2.css");
103     let mut readvec: Vec<u8> = vec![];
104     std::fs::File::open(&dir).unwrap().read_to_end(&mut readvec).unwrap();
105     dir.pop();
106     dir.push("index.css");
107     let mut file: File = std::fs::OpenOptions::new().read(true).write(true).truncate(true).open(&dir)
108     .unwrap();
109     file.write_all(&readvec).unwrap();
110
111     Status::Ok
112 }
```

POST method endpoint.

```
126 #[post("/admin/login", data = "<data>")]
127 async fn adminlogin_post(jar: &CookieJar<'_>, db: &Logs, data: Form<LoginData>) -> Status {
128     let data: LoginData = data.into_inner();
129     if data.username == "admin" && data.password == "admin" {
130         let cookie_id: String = util::generateCookie();
131         let mut cookie: Cookie<'_> = Cookie::new("admin", cookie_id.clone());
132         cookie.set_path("/admin");
133         jar.add(cookie);
134
135         let _ = sqlx::query(sql::CREATE_SESSION)
136             .bind(&cookie_id)
137             .bind(&cookie_id)
138             .bind((util::timeNow() as i64 + SESSIONTIME as i64))
139             .execute(&**db).await;
140         return Status::Ok;
141     }
142     Status::Unauthorized
143 }
```


GET method endpoint.

```
179   #[get("/admin", rank = 2)]
180   async fn admin2() -> Redirect {
181       Redirect::to(uri!("/admin/login"))
182   }
```

12. Generating HTML chat messages

When a chat message is received, the frontend JavaScript receives the message and has to display it. In order to achieve this, the script copies an existing template which is meant to be used for chat messages, fills it out with the received data and places it in the chat box. Additionally, each message is given a unique UUID such that they can be tracked by the frontend scripts and referenced later.

```
6   function addMessage(author, content) {
7       let newtempl = TEMPLATE.content.cloneNode(true)
8       newtempl.querySelector("span").innerHTML = author + ":"
9       newtempl.querySelector("p").innerHTML += (" " + content)
10      newtempl.id = self.crypto.randomUUID()
11      document.getElementById("inputform").before(newtempl)
12  }
```

13. Template generation

As a way of displaying up to date information regarding the live stream such as the title and description the program utilizes template generation which works by inserting certain values into the HTML code as well as modifying the HTML code depending whether certain conditions are met, such as modifying whether the Login/Sign up buttons should appear or if the “Logout” button should be present depending on whether the user was authenticated successfully or not.

```
7       {% if logged_in %}
8           <a class="navbar-item" onclick="logout()">Logout</a>
9       {% else %}
10          <a class="navbar-item" href="/login">Login</a>
11          <a class="navbar-item" href="/register">Register</a>
12      {% endif %}
```